CMPUT 461/561 Fall 2026

Lec 02 -- Regular Expressions, FSA, Transducers

© Denilson Barbosa



Regular Expressions and Automata

- Regular expressions
- Capture groups, memory
- Finite State Automata
- Non-determinism
- Regular languages and the Chomsky hierarchy

Regular Expressions

> “A **regular expression**, regex or regexp ... is a sequence of characters that define a **search pattern**. Usually such patterns are used by string searching algorithms for “find” or “find and replace” operations on strings, or for input validation.”

> “Regular expressions originated in 1951, when mathematician Stephen Cole Kleene described **regular languages** using his mathematical notation called regular events. These arose in theoretical computer science, in the subfields of **automata theory** (models of computation) and the description and classification of formal languages.”

Search patterns. (a.k.a. regular expressions)

Notation: regular expressions will always be enclosed by `/` and `/`

The simplest **search pattern** is one that contains the exact string being searched.

Examples

pattern	match
`/woodchuck/`	would a **woodchuck** chuck wood?
`/a/`	M**a**ry Ann

> The notion of **matching** depends on the tool being used (e.g., a text editor vs a command like tool like `grep`).

Search patterns

RE	Example Patterns Matched
/woodchucks/	"interesting links to woodchucks and lemurs"
/a/	"Mary Ann stopped by Mona's"
/!/	"You've left the burglar behind again!" said Nori

Figure 2.1 Some simple regex searches.

QUESTION: why didn't `/a/` match "...Mon**a**'s"?

Disjunctions

A pair of brackets `[ ]` can be used to specify a disjunction of characters to match.

- For example, the pattern `/[wW]/` matches either **w** or **W**.
- Similarly, `/[wW]oodchuck/` matches either **woodchuck** or **Woodchuck**.

Character ranges

Square brackets can also be used to specify a range of values (note the `-`):

- The pattern `/[2-5]/` specifies any one of the characters 2, 3, 4, or 5.
  - Note the range in this example is a shorthand for `/[2345]/`
- The pattern `/[b-g]/` specifies one of the characters b, c, d, e, f, or g.

Negation

A caret (`^` - circumflex) appearing as the **first character** inside brackets is used to negate the pattern inside the brackets.

A caret anywhere else is treated as a normal character

RE	Match (single characters)	Example Patterns Matched
/[^A-Z]/	not an upper case letter	"Oyfn pripetchik"
/[^Ss]/	neither 'S' nor 's'	"I have no exquisite reason for't"
/[^.]/	not a period	"our resident Djinn"
/[e^]/	either 'e' or '^'	"look up ^ now"
/a^b/	the pattern 'a^b'	"look up a^ b now"

Figure 2.4 The caret ^ for negation or just to mean ^. See below re: the backslash for escaping the period.

Aliases

Note that multiple ranges can appear together (ex: `[a-zA-Z0-9]` matches a single character from any of the three ranges (`a-z` , `A-Z` , or `0-9`))

RE	Expansion	Match	First Matches
\d	[0-9]	any digit	Party␣of␣5
\D	[^0-9]	any non-digit	Blue␣moon
\w	[a-zA-Z0-9_]	any alphanumeric/underscore	Daiyu
\W	[^\w]	a non-alphanumeric	!!!!
\s	[␣\r\t\n\f]	whitespace (space, tab)	
\S	[^\s]	Non-whitespace	in␣Concord

Figure 2.7 Aliases for common sets of characters.

Optional matches

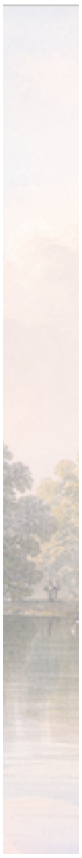
op	meaning	example
`.`	matches any character except `\n`	`/wo.dchuck/`
`?`	makes previous sub-expression optional	`/woodchucks?/`
`*`	matches the previous sub-expression 0 or more times	`/wo*dchuck/`
`+`	matches the previous sub-expression 1 or more times	`/wo+dchuck/`

The `*` is called the "**Kleene star**" operator. Similarly, the `+` is sometimes called the "Kleene +".

Exact "counting"

RE	Match
*	zero or more occurrences of the previous char or expression
+	one or more occurrences of the previous char or expression
?	exactly zero or one occurrence of the previous char or expression
{n}	*n* occurrences of the previous char or expression
{n,m}	from *n* to *m* occurrences of the previous char or expression
{n,}	at least *n* occurrences of the previous char or expression
{,m}	up to *m* occurrences of the previous char or expression

Figure 2.8 Regular expression operators for counting.



Regular expressions cannot (really) count

Note that `{n}` expressions allow us to specify a **fixed** number of repetitions:

- Ex: `/a{5}ba{5}/` matches one `b` "surrounded" by 5 `a`s on both sides.

In the study of formal languages, **counting** means checking if one character appears the same number of times as another character.

- Ex: checking for **balanced brackets** (i.e., that the same number of `(` and `)` appear in a math expression), for any number of brackets.

> An important result in formal language theory is that **regular languages** cannot count. Therefore, regular expressions, FSA, and transducers, cannot count.

Anchors

A caret `^` at the start of a regular expression matches the beginning of the string.

- Ex: `/^be/` matches **be**, **be**t, and **be**at, but not a**be**.

The dollar sign `$` matches the end of the string.

- Ex: `/be$/` matches **be**, a**be**, and cu**be**, but not **be**e.

Anchors are often used together to fully delimit a pattern.

- Ex: `/^.*be.*$/` matches all strings that contain **be** anywhere.

Disjunction of sub-expressions

Recall that we used lists of characters inside brackets `[ ]` to indicate disjunction.

- And that it worked for single characters only.

The pipe `|` is also used for disjunction, but this time for longer patterns.

Example:

`/cat|dog/` matches either the word **cat** or the word **dog**.

Scope of operators

Parentheses `( )` are used to change the scope of the operators:

`/gupp(y|ies)/` matches either guppy or guppies because the disjunction is defined only over the sub-expressions within parenthesis

`/Chapters?\s([0-9]{1,2},?)+/` matches strings that specify either one chapter number or a list of two-digit chapter numbers.

- Ex: "Chapter 1" and "Chapters 31,09" are both matched.

Escaping

The backslash, `\`, is used to escape special characters:

RE	Match	First Patterns Matched
*	an asterisk "*"	"K*A*P*L*A*N"
\.	a period "."	"Dr. Livingston, I presume"
\?	a question mark	"Why don't they come and lend a hand?"
\n	a newline	
\t	a tab	

Figure 2.9 Some characters that need to be backslashed.



Building regular expressions

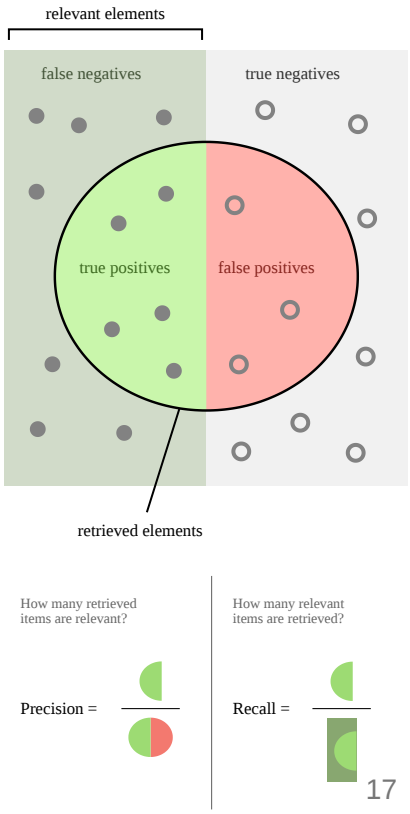
- `https://regex101.com/` can come in handy

Building regular expressions is an iterative process

Often, we:

1. Start with the regex we think will work. Then:
2. If it is too restrictive, we relax it to get more matches (i.e., increase recall). Or,
3. If it is too permissive, make it stricter to get fewer false-positive matches (i.e., increase precision).

This precision vs recall trade-off is ever present when working on all NLP tasks, BTW.



Example: matching occurrences of `the`...

	pattern	permissive/restrictive?	why?
step 1	`/the/`	restrictive	fails to match capitalized words
step 2	`/[tT]he/`	permissive	matches words like theory
step 3	`/\W[tT]he\W/`	...	...

Should we stop after step 3 or keep going?

- It depends... have you achieved a good precision/recall balance on your test data?

Aside: some systems allow the shortcut `\b` to match a word boundary (of any kind).



Using regular expressions

- Python `re` module

`https://docs.python.org/3/library/re.html`

Basic usage

Returns the first match of a pattern within a string:

```
result = re.match(pattern, string)
```

Compiles the pattern into a reusable object (faster for several matches):

```
prog = re.compile(pattern)
result = prog.match(string)
```

Python offers different primitive operations based on regular expressions:

- `re.match()` checks for a match only at the beginning of the string
- `re.search()` checks for a match anywhere in the string (like Perl)
- `re.fullmatch()` checks for entire string to be a match

Finding all matches

As discussed earlier, most regular expression systems return only the **first match**.

In Python, we can gather all matches using `re.findall()`

```
re.findall(r'\bf[a-z]*', 'which foot or hand fell fastest')
['foot', 'fell', 'fastest']

# finding all adverbs (?!?!?!)
text = "He was carefully disguised but captured quickly by police."
re.findall(r"\w+ly\b", text)
['carefully', 'quickly']
```


Substitutions and capture groups

UNIX tools (ex `sed`) supported substitution commands like `s/pattern/substitution/` to be applied on text.

- They worked by replacing matches of the pattern by the substitution string.

They were especially useful with **capture groups**, which act like registers that keep the matches of the patterns within parenthesis.

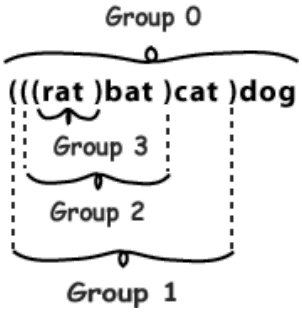
If you don't want the value matched by an expression within parentheses to be captured, define the sub-expression as non-capturing: `/... (:?... ).../`

Nested capture groups

`/(((rat )bat )cat )dog/`

string: rat bat cat dogwood

NOTE: the example should have had `\s` for blank spaces!



https://chortle.ccsu.edu/finiteautomata/Section09/sect09_12.html

Nested capture groups in Python

```
>>> p = re.compile('(a(b)c)d')
>>> m = p.match('abcd')
>>> m.group(0)
'abcd'
>>> m.group(1)
'abc'
>>> m.group(2)
'b'
```

Examples from ELIZA

ELIZA applied simple substitution rules to obtain responses to user input:

- It did not keep track of previous interactions.
- Nevertheless, the substitutions were still enough to give the illusion of a conversation with a psychiatrist.

```
s/.* I'M (depressed|sad) .*/I AM SORRY TO HEAR YOU ARE \1/
s/.* I AM (depressed|sad) .*/WHY DO YOU THINK YOU ARE \1/
s/.* all .*/IN WHAT WAY/
s/.* always .*/CAN YOU THINK OF A SPECIFIC EXAMPLE/
```




Automata

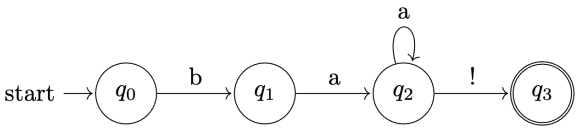
- Finite-state Automata (FSA)
- Non-determinism
- Regular expressions and FSA
- Regular languages and the Chomsky hierarchy

Example Finite-state Automaton



Imagine we want to build a machine to **recognize** sheep-talk.

Language Detection task.



The machine operates by "looking" at each character of what is being said.

It will **accept** the input string if it is sheep-talk...

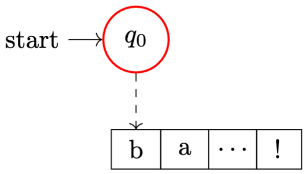
... and reject any other kind of input.

How does a deterministic FSA work?

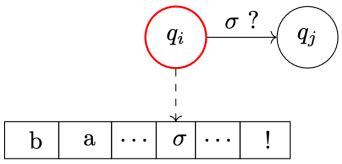
It is always in **one** state (q_0, q_1, ...).

It always reads **one symbol** at a time.

It always **starts** in the same state, reading the first symbol in the string.



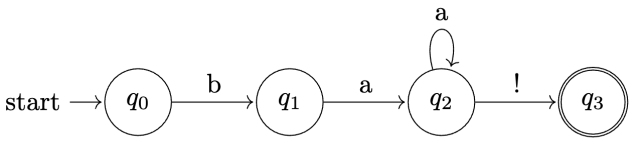
It **transitions** from state q_i to q_j if there is an arc labeled with the symbol being read at that time. If so, the FSA moves to the next symbol.



It **acepts** the string if it arrives at an accepting state (double lines) after reading the last symbol.

Example: accepting `baaa!`

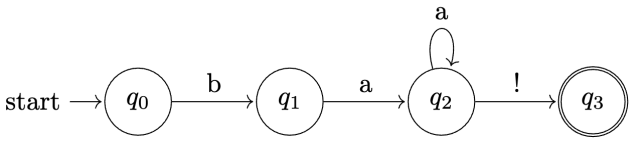
state	symbol	transition to?
q_0	b	q_1
q_1	a	q_2
q_2	a	q_2
q_2	a	q_2
q_2	!	q_3



Because we end up in q_3 after reading the last symbol and q_3 is accepting, **the string is accepted**!

Example: rejecting `boo!`

state	symbol	transition to?
q_0	b	q_1
q_1	o	✗



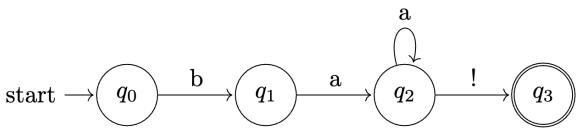
Since the automaton **does not** reach the last symbol, the **string is rejected**.

Formal definition

A Deterministic Finite-state Automaton (DFA) is a 5-tuple consisting of:

- A set of states Q
- An alphabet (set of symbols) Σ
- An initial state $q_0 \in Q$
- A set of accepting states $F \subseteq Q$
- A transition function $\delta : Q \times \Sigma \to Q$

Example:



- $Q = \{q_0, q_1, q_2, q_3\}$
- $\Sigma = \{\mathrm{a}, \mathrm{b}, !\}$
- Initial state= q_0
- $F = \{q_3\}$
- $\delta = (q_0, \mathrm{b}) \to q_1; (q_1, \mathrm{a}) \to q_2\ (q_2, \mathrm{a}) \to q_2; (q_2, !) \to q_3$

Formal definition of acceptance

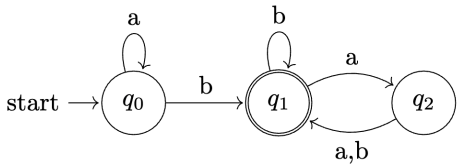
Given a DFA $D = (Q, \Sigma, q_0, F, \delta)$ and a string $s = s_0\, s_1\, \cdots\, s_n$ (with n symbols), we simulate F from the start state, transitioning states according to the characters in s.

The automaton is said to **accept** the string if (and only if) the state it is in after processing all characters in s is an accepting state:

$$\delta(\delta(\cdots\delta(\delta(q_0, s_0), s_1)\cdots), s_n) \in F$$

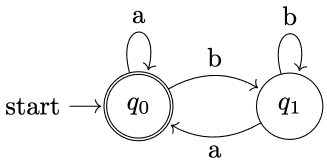
The set of strings accepted by D is called the **language** of D.

Which strings are accepted?



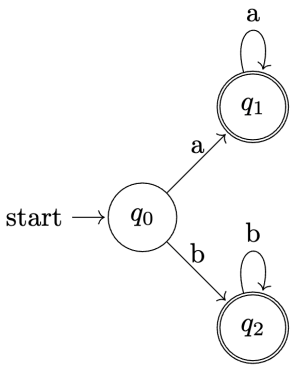
1. ϵ (the empty string)
2. a
3. ab
4. aba
5. abbbb
6. baaa

Which strings are accepted?



1. ϵ (the empty string)
2. a
3. ab
4. aba
5. abbbb
6. baaa

Which strings are accepted?



1. ϵ (the empty string)
2. a
3. ab
4. aba
5. abbbb
6. baaa

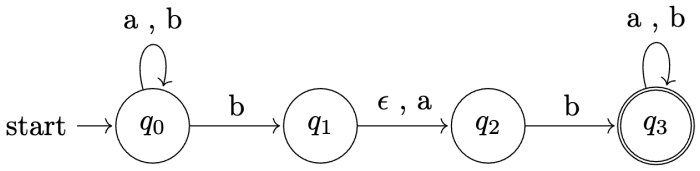


Non-determinism

- Non-deterministic FSA
- Union, intersection, concatenation operations

Non-deterministic FSA

A non-deterministic finite-state automaton (NFA) is an automaton in which multiple choices exist for the next state, given the current state and the input.



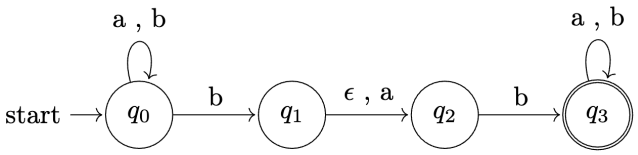
When in q_0 and reading a `b` this NFA may

- Remain in q_0
- Transition to q_1
- Transition to q_2 (via the ϵ transition)

While in q_2, this NFA may transition to q_3:

- By reading an `a`
- Without reading any symbol!

Which strings are accepted?



you can also take alternative paths
ie. exhaust all possible paths

1. ϵ (the empty string)
2. a
3. ab
4. aba
5. abbbb
6. baaa

3,4,5,6?

Formal definition of NFA

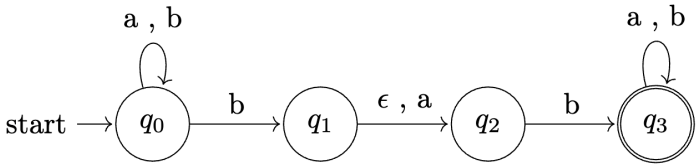
A Non-deterministic Finite-state Automaton (NFA) is a 5-tuple consisting of:

- A set of states Q
- An alphabet (set of symbols) Σ
- An initial state $q_0 \in Q$
- A set of accepting states $F \subseteq Q$
- A transition function $\delta : Q \times \Sigma \cup \{\epsilon\} \to \mathcal{P}(Q)$

Here $\mathcal{P}(Q)$ means the **power set** of Q.

Note that the transition function may "consume" a symbol from Σ or the empty string ϵ.

Example NFA



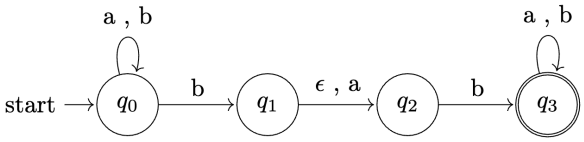
$Q = \{q_0, q_1, q_2, q_3\}$
$\Sigma = \{\mathrm{a}, \mathrm{b}\}$
Initial sate: q_0
$F = \{q_3\}$

$\delta(q_0, \mathrm{a}) = \{q_0\}$
$\delta(q_0, \mathrm{b}) = \{q_0, q_1, q_2\}$
$\delta(q_1, \mathrm{a}) = \delta(q_1, \epsilon) = \{q_2\}$
$\delta(q_2, \mathrm{b}) = \{q_3\}$
$\delta(q_3, \mathrm{a}) = \delta(q_3, \mathrm{b}) = \{q_3\}$

Acceptance with an NFA

Because of the non-determinism, the same string can be "consumed" through different sequences of states by the NFA.

Example: different state sequences while processing `abb`:

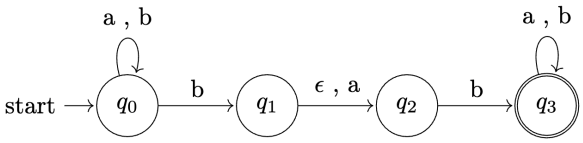


$q_0 \xrightarrow{a} q_0 \xrightarrow{b} q_0 \xrightarrow{b} q_1$ (non-accepting)

$q_0 \xrightarrow{a} q_0 \xrightarrow{b} q_2 \xrightarrow{b} q_3$ (accepting, note the ϵ-transition from q_1)

We say that an NFA F accepts a string s if **there exists one sequence of transitions** that "consumes" s and ends in an accepting state.

Equivalent states



Two (or more) states that can be reached by the NFA while reading the same (sub)string are **equivalent**.

Ex: q_0, q_1 and q_2 all can be reached from q_0 itself when we read a `b`.

Note that all pairs of states connected by ϵ-transitions are, by definition, equivalent.

Relationship between DFA and NFA

Every DFA is also an NFA. By definition, a DFA is a special case of an NFA.

Every NFA can be rewritten as a DFA. There is a deterministic algorithm to rewrite any NFA into an equivalent DFA (see CMPUT474 if interested).

Why does this matter?
You can think of non-determinism as, essentially, a more convenient way of writing automata: they can be composed (via concatenation, union, or looping) using ϵ-transitions.

More formal statements

These are the subject of CMPUT 474:

1. The set of regular languages is **closed** under union and concatenation.
2. Every NFA is equivalent to some DFA.
3. Multiple NFA (or even DFA) can accept the same language.
4. NFA and DFA have the same computing power.
5. Languages accepted by NFA (or DFA) are all regular.
6. NFA and DFA cannot count.
    - There is a formal proof of this statement (the Pumping Lemma).

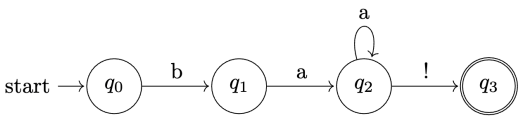
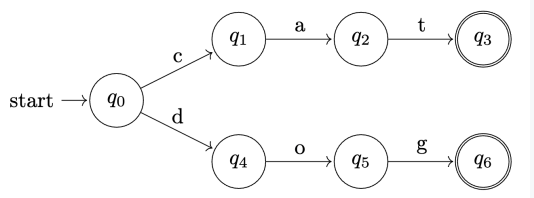


Regular Expressions and FSA

- Converting Regular Expressions into NFA (and vice-versa)

How are regular expression related to FSA?

It is easy to see that we can come up with FSA that recognize exactly the set of strings matched by some regular expressions. But is that always true?

`/ba+!/`	
`/cat|dog/`	

Equivalence between regular expressions and FSA

We can always convert a regular expression into an FSA (and vice-versa). It is easier to show that using NFA. While the formal proof is covered in CMPUT 474, here is a hint:

The **basic idea** is that regular expressions are built out of the following operations:

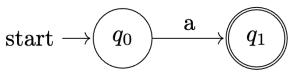
- **concatenation**; matching `/ba/` means matching `/b/` followed by matching `/a/`
- **union** (disjunction); matching `/b|a/` means matching `/b/` or `/a/`
- **repetition**; matching `/a*/` is a loop that matches `//` (ϵ) then `/a/`
- **negation**: matching `/[^a]/` means rejecting a match of `/a/`

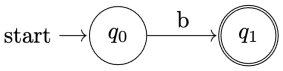
Note that other constructs can be rewritten using the operations above (ex: `/a+/` = `/aa*/` and `/a?b/` = `/b|ab/`)

Equivalence between regular expressions and FSA

The rest of the proof uses the following ideas:

For each symbol in a regular expression, we can build an NFA that accepts only that symbol:

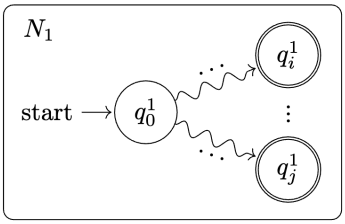
a	

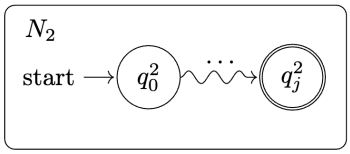
b	

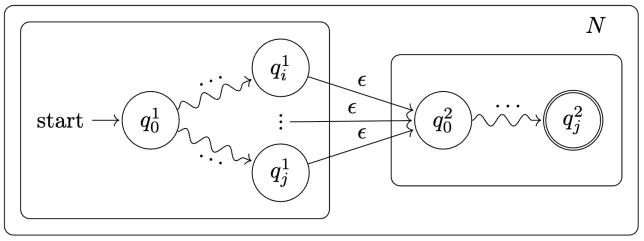
Now, all that we need is to show how to build the NFA corresponding to each of the operations (concatenation, union, repetition, negation)!

NFA concatenation

Use ϵ transitions to connect the accepting states of the first NFA into the start state of the second NFA!







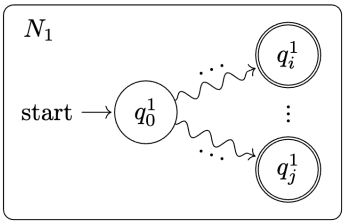
Ex:

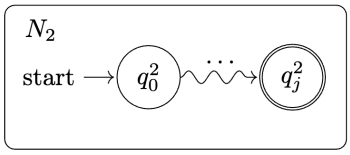
N_1 accepts `baaa`, N_2 accepts `boo!`

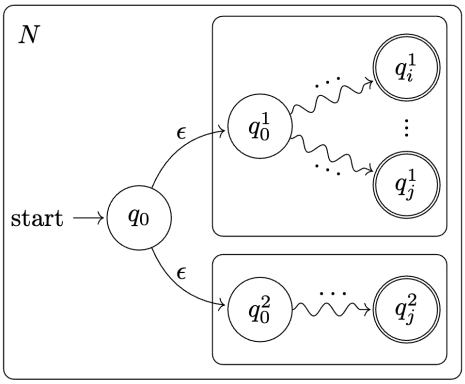
N accepts `baaaboo!`

NFA union

Using ϵ transitions, connect a "dummy" start state to the start states of the two NFA.





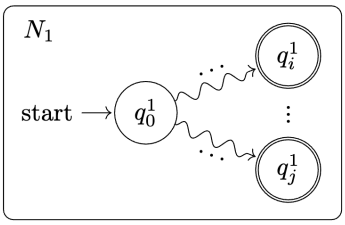


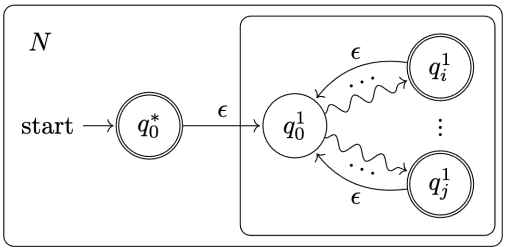
Ex: N_1 accepts `baaa`, N_2 accepts `boo!`

N accepts `baaa` or `boo!`

NFA Kleene star

We need a new start state, which is also accepting (so that we can accept ϵ) and we need to loop back from every original accepting stating (so we can accept one more string).





NFA negation

The negation of a regular expression e matches every string that is **not matched** by e itself.

We can get the FSA that computes the negation as follows:

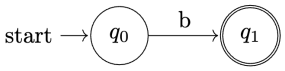
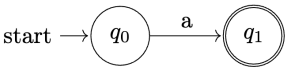
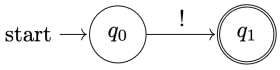
1. Convert e into an NFA N_1 (!)
2. Convert N_1 into a minimal equivalent DFA D_1
3. Create a DFA D_2 that is the **complement** of D_1: a "copy" of D_1 where we "flip" accepting and non-accepting states and add new transitions that complement the original ones.

While this works, negation is a problematic operation to begin with.

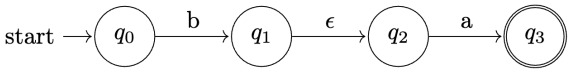
- Should the regular expression `/[^a]/` match the string `b`???

Example: the NFA for `/ba+!/` = `/baa*!/`

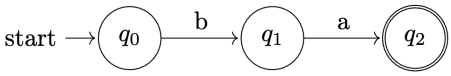
Building blocks:

b		a		!	

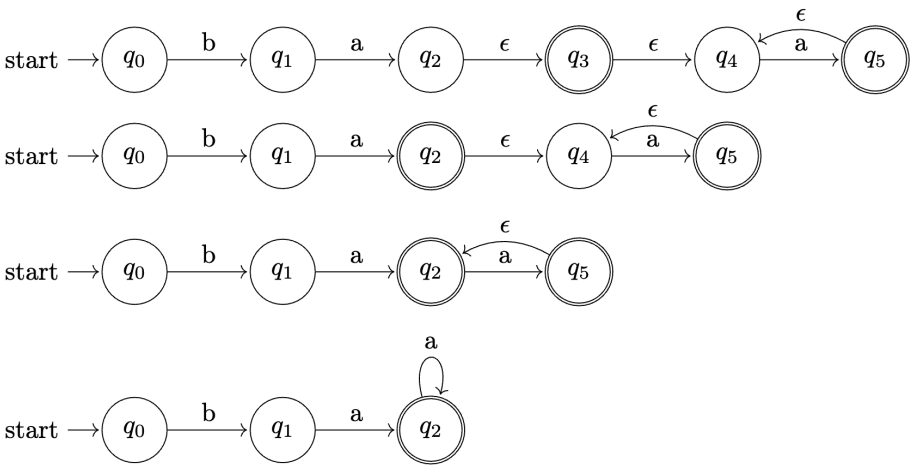
Concatenating `/b/` and `/a/`:



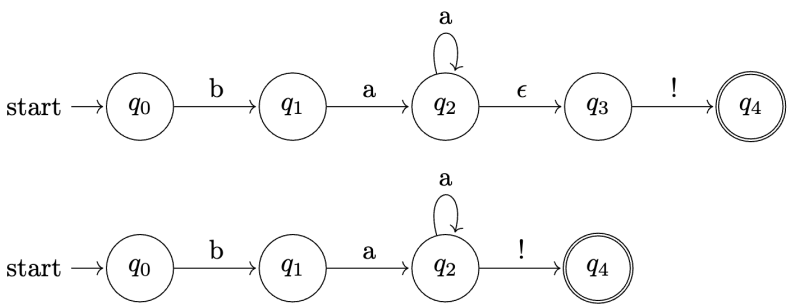
We can simplify the NFA above by removing the transition with ϵ alone!



Concatenating `/ba/` and `/a*/`, followed by ϵ-transition simplification:



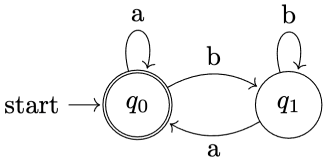
Finally, we can concatenate `/baa*/` with `/!/`, arriving at the final simplified NFA:



Writing the regular expression of an NFA

One can write down a regular expression that matches the strings accepted by an NFA by recursively traversing the graph of the NFA, taking into account that:

1. A "loop back" in the NFA corresponds to a Kleene star (or variant) on a sub-expression;
2. A "fork" in the NFA corresponds to a disjunction of sub-expressions.



At q_0 we have a choice:

- "stay" (`a*`)
- "move and come back (`b+a`)

All combined: `/(a*|(b+a))*/`

More formal statements

1. Regular expressions are "equivalent" to NFA
    - We can always derive an NFA that accepts the strings matched by the regular expression.
    - We can always write a regular expression that matches the strings in the language of an NFA

Regular languages

Languages accepted by a DFA or an NFA contain only strings that can be matched by an equivalent regular expression and vice versa.



The Chomsky hierarchy

- Some languages are not regular
    - We need more powerful computing models to process them

Natural languages are not regular

We already saw a **language that requires counting** (ex: all strings with the same number of `(` and `)` symbols) and **is not regular**.

What about natural languages? Are there phenomena that make them non-regular?

Generally, anything requiring **recursion** makes the language not regular.

> **Who ate the malt?**
>
> The mouse the cat the dog chased killed ate the malt.

Centre embedding

> In linguistics, **centre embedding** is the process of embedding a phrase in the middle of another phrase of the same type. ... The most frequently used example involves embedding a relative clause inside another one.

Example:

- The mouse ate the malt.
- The mouse **the cat killed** ate the malt.
- The mouse the cat **the dog chased** killed ate the malt.
- The mouse the cat the dog **the crow saw** chased killed ate the malt.

Centre embedding and the need for recursion

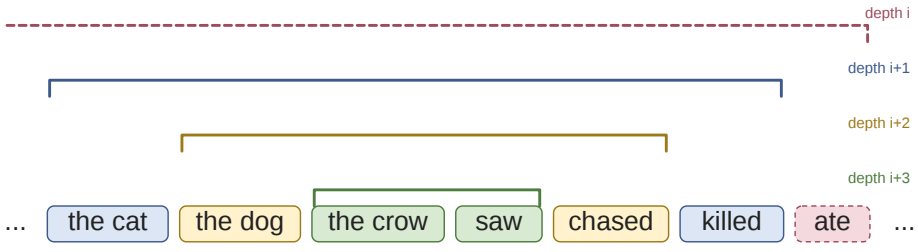
In theories of **natural language parsing**, the difficulty with multiple center embedding is thought to arise from limitations of the human short term memory.

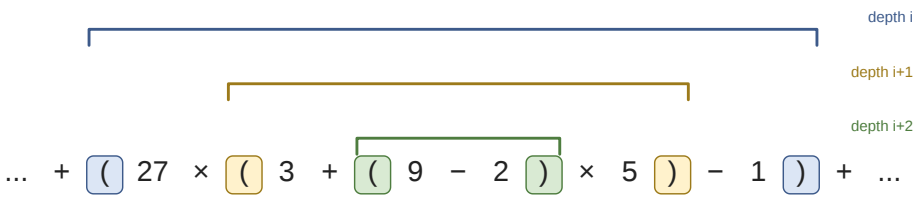
In order to process multiple center embeddings, **we have to store many subjects in order to connect them to their predicates**.

NOTE that these sentences are grammatical and perfectly acceptable in English.

However, they are **inherently recursive**: we need to parse each "nested phrase" separately to make sense of the entire sentence.

Parsing centre embedding and counting both need recursion



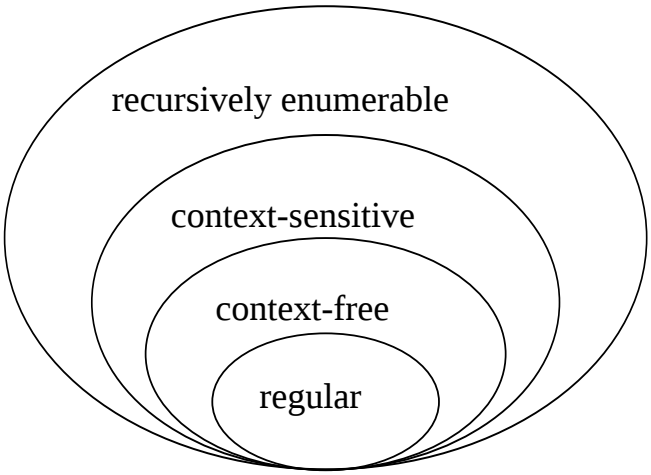


The Chomsky hierarchy

Avram Noam Chomsky is an American professor and public intellectual. Sometimes called "the father of modern linguistics", he is also known for his political activism and social criticism.

Chomsky proposed a strict hierarchy of languages based on their complexity.

Most phenomena occurring in natural (and computer) languages fall within context-free languages.



ASIDE: Languages and automata

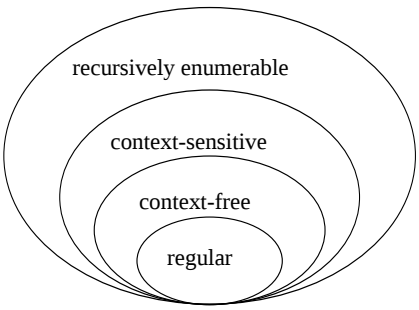
A language is a **set of strings**.

An automaton is an abstract computation model (a machine).

- Input: string
- Output: accept/reject

Each automaton defines a language: the set of strings that it accepts. For each language there is an automaton that accepts it.

Automaton "power" is inherently associated with language complexity.



Regular ⇔ NFA
Context-free ⇔ Push-down automata
Cont.-sensitive ⇔ restricted Turing Machine
Rec. enumerable ⇔ full Turing Machine

ASIDE: Natural Languages and automata

Regular

- Ex: $\{a^n | n > 0\}$.
- Can be recognized by DFA/NFA.
- "Correspond" to regular expressions.

Context-free

- Ex: $\{a^n b^n | n > 0\}$.
- Require push-down automata (i.e. **recursion**).

Capture pretty much all natural language and programming language phenomena.

Corollary: while FSA/regular expressions are sufficient for many tasks (ex: data cleaning, recognizing simple languages) **we need more power** (i.e., recursion) for most tasks (ex: proper parsing).

Centre embedding and Claude

D Who loves the toy? The toy the man the woman the kid the dog the cat avoided followed heard saw loves.

Let me parse this center-embedded sentence step by step:

"The toy the man the woman the kid the dog the cat avoided followed heard saw loves."

Working through the nested structure:

- The cat avoided (the dog)
- The dog followed (the kid)
- The kid heard (the woman)
- The woman saw (the man)
- The man loves (the toy)

So **the man** loves the toy.

The sentence chains together multiple relationships, but when we trace through to find who performs the final action of "loving," it's the man who loves the toy.

Centre embedding and ChatGPT

Who loves the toy?
The toy the man the woman the kid the dog the cat avoided followed heard saw loves.

Thought for 6s

The man.

Parsed with the relative clauses:
The toy **[that the man [that the woman [that the kid [that the dog [that the cat avoided] followed] heard] saw] loves]**.

Reading the embeddings outward: the cat avoided the dog → the dog followed the kid → the kid heard the woman → the woman saw the man → the man loves the toy.

Centre embedding and (some online demo of) LLama

Who loves the toy? The toy the man the woman the kid the dog the cat avoided followed heard saw loves.

9/10/2025, 3:13:04 PM

Llama

Another clever twist on the classic tongue-twister!

Let's analyze the sequence of events:

1. The toy was loved by a man.
2. The toy was the woman's (the wife's) toy.
3. The toy was the kid's (the child's) toy.
4. The toy was the dog's toy.
5. The toy was the cat's toy.
6. The toy was avoided by all.
7. The toy was followed by the heard (a dog) and the saw (a bird) and the loved (a cat) and the had (a child) and the saw (a bird) and the loved (a cat) and the had (a child) and the saw (a bird) and the loved (a cat) and the had (a child) and the saw (a bird) and the loved (a cat) and the had (a child) and the saw (a bird) and the loved (a cat) and

Llama-3.2-1B-Instruct-q4f32_1-MLC



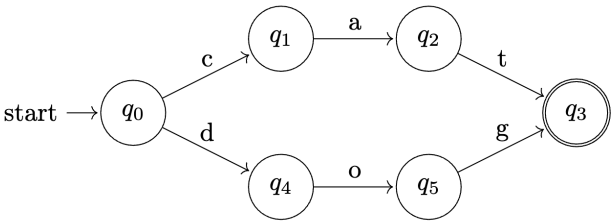
Transducers and early NLP tools

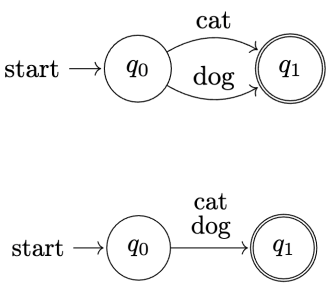
- Transducers
  - Early word recognizers and part-of-speech taggers

Notation

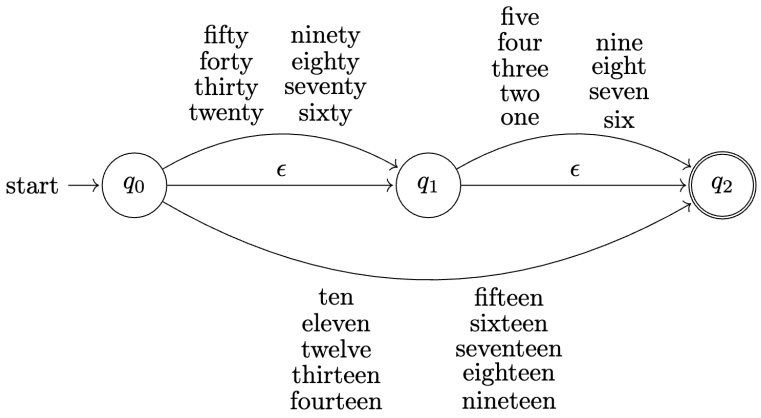
FSA can grow very big even when they are used to simply recognize words.

To alleviate that problem, we will use full words instead of individual symbols and stack multiple words/symbols on top of one another:



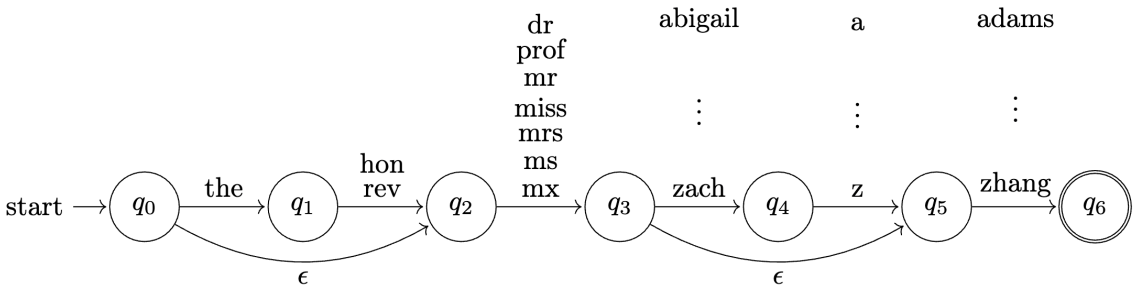


FSA as a "shallow grammar" for numbers



FSA as a "shallow grammar" for personal titles

Sketch of a machine to recognize lowercased English names with some common titles:



https://en.wikipedia.org/wiki/English_honorifics

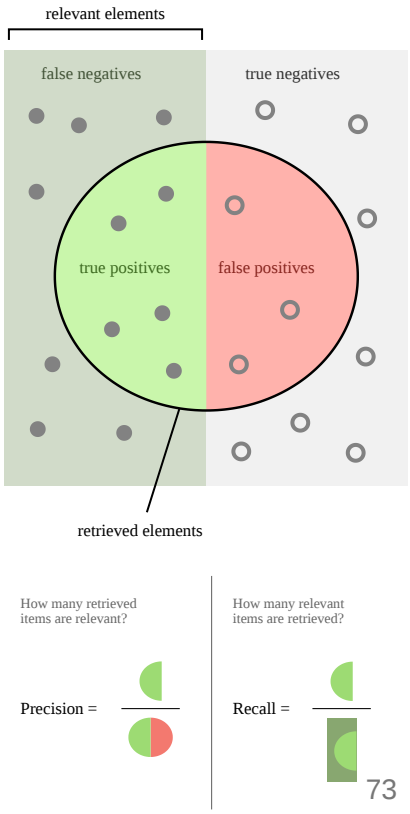
Will that FSA Recognize all Proper Names?

If we want to extract all the proper names in the news, will this work?

- What will it miss?
- Will it accept something that is not a proper name?
- How would you change it to accept all proper names without false positives?

When do you stop modifying it and move it into "production"?

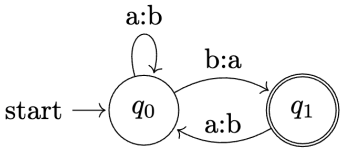
Precision vs. recall….



Transducer

A Finite-state **Transducer** is a 7-tuple $T = (Q, \Sigma, \Delta, q_0 \in Q, F \subseteq Q, \delta, \sigma)$ where Q, Σ, q_0 F and δ are exactly as before.

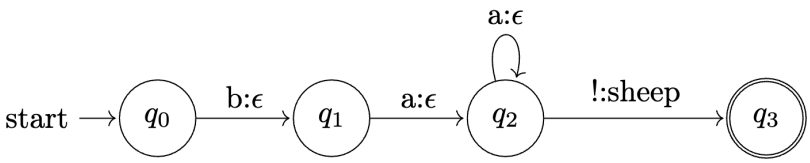
- Δ is the **output alphabet**
- $\sigma : Q \times \Sigma \rightarrow \mathcal{P}(\Delta)$ is the output function, giving the strings that are output at each state for each input symbol



Notation: `a:b` means that the transducer outputs `b` when reading an `a`.

Ex: with input `baab`, the transducer accepts and outputs `abba`.

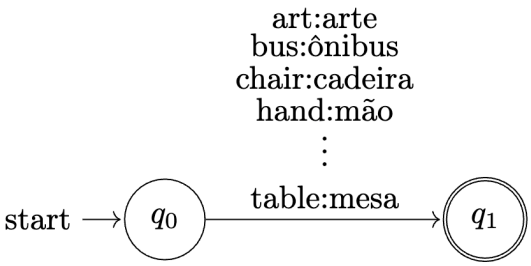
A transducer that recognizes and "tags" sheep talk!



Note that the transducer produces output only when it accepts a (sheep-talk) string.

In other words, it can be used to recognize strings in that language.

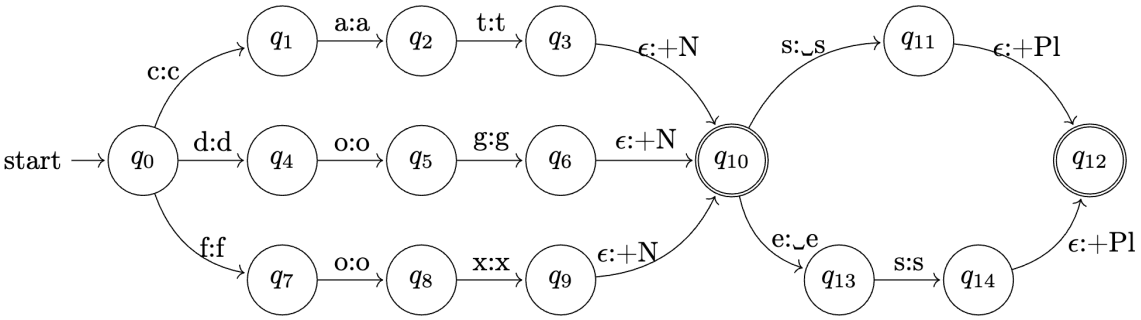
A transducer that translates some nouns into Portuguese



NOTE: many common nouns in Portuguese have gender, so a simple translation by mapping as above does not always work!

- Ex: "cat" in Portuguese is gat**a** (female) or gat**o** (male).

A transducer that recognizes nouns (singular and plural)



Examples:

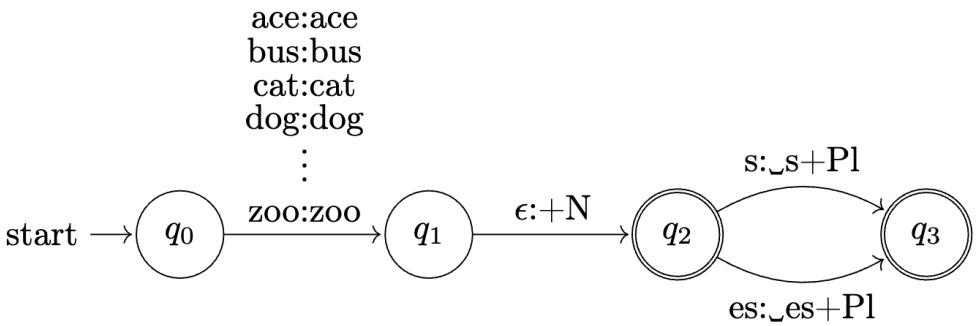
Input: `cat`
Output: `cat+N`

Input: `dogs`
Output: `dog+N` `s+Pl`

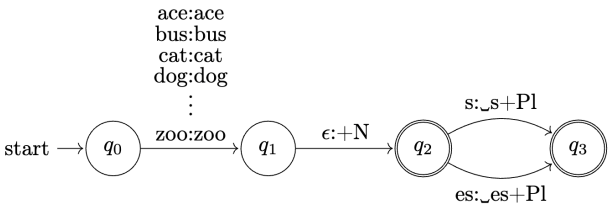
Tokenization, normalization, PoS tagging

Transducers have been used in the past to recognize and tokenize words that are members of strictly regular word classes.

For example, the following transducer recognizes quite a few nouns in their base form or in their plural form, when the plural is formed by appending `s` or `es`:



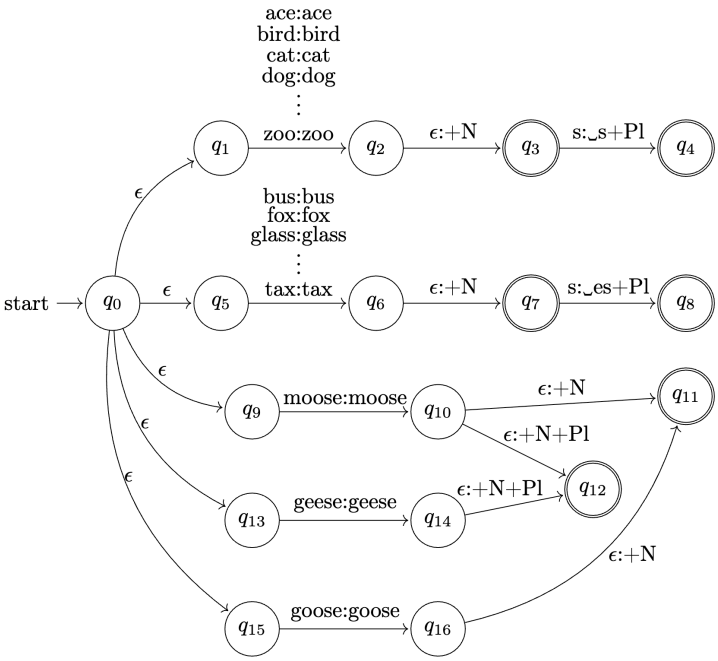
Problem #1: dealing with irregularities!



While we Transducer above correctly handles words like `cars` (output `car+N` `s+Pl`) and `buses` (output `bus+N` `es+Pl`) it would fail with:

- `cares`: even though it is a verb, the transducer would output `car+N` `es+Pl`!
- nouns with irregular plural (ex: `moose`, `goose`, ...)
- what about `fish`? Both `fish` and `fishes` are correct plural forms (although they mean different things)

Better transducer that deals with some irregularities



Problem #2: $$$

What was wrong with "classical" methods?

High expertise required

- Linguists and computer scientists earn high $$$.

Low scalability

- Systems require maintenance as language keeps evolving.

Underwhelming performance

- Natural language is very complex; systems rarely lived up to expectations.



Summary

FSA/Regex/Transducers can solve many NLP tasks!

Regular expressions and FSAs are very good at representing small (and regular) subsets of natural language.

- Suitable for well-defined tasks (ex: data cleaning, information extraction) that do not deal with all aspects of a natural language.

Can be hard to scale up: e.g., when many choices at any point (e.g. surnames)

But are very quick, powerful and easy to use for well-defined problems.

- Check out https://fomafst.github.io/ for a modern library to support FSA development.



Relationship to other modules

1. Tokenization (detecting words) and normalization (via transducers)
2. Data cleaning (ex: assignment 1)
3. Information extraction
    - extracting "regular" data like emails, phone numbers, postal codes, ...
4. Rule-based systems (classification, dialogue)
5. POS tagging via transducers

What you should know or be able to do

- Write regular expressions for tasks such as information extraction
- Convert regular expressions into DFAs
- Convert DFAs into regular expressions
- Identify inputs that are (or are not) recognized by an FSA and explain why
- Identify strings that are (or are not) matched by regular expressions and explain why
- Explain limitations of regular languages and features of natural languages that are beyond regular languages
- Explain whether or not (and why/why not) regular expressions could be used for downstream tasks (ex: intent classification).
- Explain why modern NLP tools for large-scale tasks use other technologies.